



Nguyễn Trần Hoàng Liều - 2452832

## Linear Regression

• Model equation:  $\hat{y}(x, w) = w^T x = \sum_{d=0}^{D-1} w_d x_d$

• bias trick: Set  $x_0 = 1$  so  $w_0$  (intercept) is absorbed into the vector dot product.

### Matrix representation

• design matrix:  $X \in \mathbb{R}^{N \times D}$  (row is  $x_N^T$ )

• target vector:  $t \in \mathbb{R}^{N \times 1}$

• Weight:  $w \in \mathbb{R}^{D \times 1}$

• Generative assumption:  $t = h(x) + \epsilon$ ,  $\epsilon \sim \mathcal{N}(0, \sigma^2)$

•  $h(x) \approx w^T x$  is the underlying true function

•  $\epsilon$  is zero mean gaussian noise with variance  $\sigma^2$

### Maximum Likelihood Estimation (MLE)

• Conditional likelihood for one point:  $p(t | x, w, \beta) =$

$$\left(\frac{\beta}{2\pi}\right)^{1/2} \exp\left(-\frac{\beta}{2}(t - w^T x)^2\right) = \mathcal{N}(t | w^T x, \beta^{-1})$$

• Negative Log-likelihood (NLL):  $L(w, \beta) = \beta E_\beta(w) - \frac{N}{2} \ln(\beta)$

$$SSE(E_\beta) = \frac{1}{2} \sum_{n=1}^N (t_n - w^T x_n)^2 \parallel + \frac{N}{2} \ln(2\pi)$$

➔ Minimizing NLL with respect to  $w$  is identical to minimizing the Least Squares Error ( $E_\beta(w)$ )

## B. Mini batch GD

Used when:  $N$  or  $D$  is huge

$$\text{batch loss: } \bar{E}_D(w) = \frac{1}{2B} \sum_{n=1}^B (t_n - w^T x_n)^2$$

$$\text{Gradient } \Delta w_B = \frac{1}{B} \sum_{n=1}^B (w^T x_n - t_n) x_n$$

$$\text{Update Rule: } w^t = w^{(t-1)} - \eta \Delta w_B \quad (\eta: \text{learning rate})$$

## C. Model Evaluation:

Prediction:  $\hat{y}_{\text{pred}} = \sum w_{ML}$  and residual vector  $e = t - \hat{y}_{\text{pred}}$

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N (t_n - \hat{y}_n)^2 \quad (\text{measure variance of error})$$

$$\text{RMSE} = \sqrt{\text{MSE}} \quad (\text{std of error, same unit as } t)$$

## D. Overfitting & Regularization

When capacity  $M$  is too high, the model fits the training noise

regularization: principle:

$$\min_w \frac{1}{2} \sum_{n=1}^N [t_n - w^T x_n]^2 \quad \text{subject to} \quad \sum_{m=1}^M |w_m|^q \leq \eta$$

Feature

Ridge Regression ( $L_2$ )

Lasso ( $L_1$ )

Norm power

$$q = 2$$

$$q = 1$$

Loss function

$$L(w) = E_D(w) + \frac{\lambda}{2} w^T w$$

$$L(w) = E_D(w) + \frac{\lambda}{2} \sum_{m=1}^M |w_m|$$

My questions:

1. Should the bias  $w_0$  be Regularized?

2. Why does the  $L_1$  penalty (Lasso) drive coefficients exactly to zero, whereas the  $L_2$  penalty (Ridge) only make them very small?

Tôn trọng - Hợp tác - Khoan dung

## Logistic Regression

Core questions: "How do we map a range of  $w^T x$  to a probability?"

### 1. Predictive model

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

Purpose: map a range of  $w^T x$  to a probability

$$z = w^T x \in \mathbb{R}$$

probability

### 2. Decision rule & bounds

$$\text{Class} = \begin{cases} C_1 & \text{if } z \geq 0 \\ C_0 & \text{if } z < 0 \end{cases}$$

Purpose: Convert the continuous output of the model into a discrete class

a discrete

threshold

linear decision boundary

### 3. Single-sample probability

$$p(y|x)$$

Purpose: Provide a formula for the probability

formula



# Logistic Regression.

Core questions: "How do we transform a linear dot product  $w^T x$  into a valid probability value in the range  $[0, 1]$ ?"

## 1. Predictive model (Sigmoid)

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + \exp(-w^T x)}$$

Purpose: maps an unbounded real-valued linear score  $z = w^T x \in (-\infty, +\infty)$  into a normalized probability  $\hat{y} = p(C_1 | x, w) \in (0, 1)$

## 2. Decision rule & boundary

$$\text{Class} = \begin{cases} C_1 (y=1) & \text{if } \hat{y} \geq \lambda \\ C_0 (y=0) & \text{if } \hat{y} < \lambda \end{cases}$$

Purpose: Convert the continuous probability output into a discrete binary decision based on a chosen threshold  $\lambda$  (commonly  $\lambda = 0.5$ , which yields the linear decision hyperplane  $w^T x = 0$ ).

## 3. Single-Sample Bernoulli Likelihood

$$p(y | x, w) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Purpose: Provides a compact, single line mathematical formula for the  $y \in \{0, 1\}$

#### 4. Joint Dataset Likelihood

$$p(t | X, w) = \prod_{n=1}^N \hat{y}_n^{y_n} (1 - \hat{y}_n)^{1-y_n}$$

→ Measure the overall probability of generating the observed labels of all  $N$  independent training samples under parameters  $w$ .

5. Loss function: Binary cross entropy, Negative log likelihood.

$$L(w) = -\sum_{n=1}^N [y_n \ln \hat{y}_n + (1 - y_n) \ln (1 - \hat{y}_n)]$$

→ Converts the product of probability into a convex minimization problem; penalizes predictions that are confident wrong.

#### 6. first-order Gradient (for GD)

$$\Delta w = \frac{\partial L}{\partial w} = X^T (\hat{y} - y) = \sum_{n=1}^N (\hat{y}_n - y_n) x_n$$

My questions:

1. Why it is called Logistic Regression when it is used for Classification?

2. Why use Sigmoid but not other step function or linear function?

4. Why use cross

⊗ Decision Tree  
Classification tree

Regression Tree

Entropy:  $H(S)$

(measure the un-

Information gain:

$S$ : dataset,  $A$ :

split.  $S_{\text{left}}$ : sub

→ Measure the

Gini Index:

. Another meas

.  $Gini(S) = 0 \Rightarrow$

ID3: use Entro

. Greedy approach

. Recursive spl

the same

Limitations: - over

- bias